

DATA STRUCTURES AND ALGORITHMS

— Hash Table —

CCE 105
Data
Structures

Prepared by:



Topics

- Hashing
- Hashing Techniques
- Collision Resolution Techniques



01

Hashing

Hash Tables

A hash table is a data structure that offers very fast insertion and searching.

For a human user of a hash table, this is essentially instantaneous. It's so fast that computer programs typically use hash tables when they need to look up tens of thousands of items in less than a second (as in spelling checkers).



Hash Tables

One important concept is how a range of key values is transformed into a range of array index values. In a hash table this is accomplished with a hash function. However, for certain kinds of keys, no hash function is necessary; the key values can be used directly as array indices.



Hashing

Hashing is the application of a mathematical function (called a hash function) to the key values that results in mapping the possible range of key values into a smaller range of relative addresses. The hash function is something like a locked box that needs a key to get the output which, in this case, is the address where the key is stored:

key ==>

Hash Function $H(k)$

==> address



Hashing

With hashing, there is no obvious connection between the key and the address generated since the function "randomly selects" an address for a specific key value, without regard to the physical sequence of the records in the file. Hence, it is also known as randomizing scheme.



Hashing

Two or more input keys, say k_1 and k_2 , when applied to a hash function, may hash to the same address, an accident known as collision.

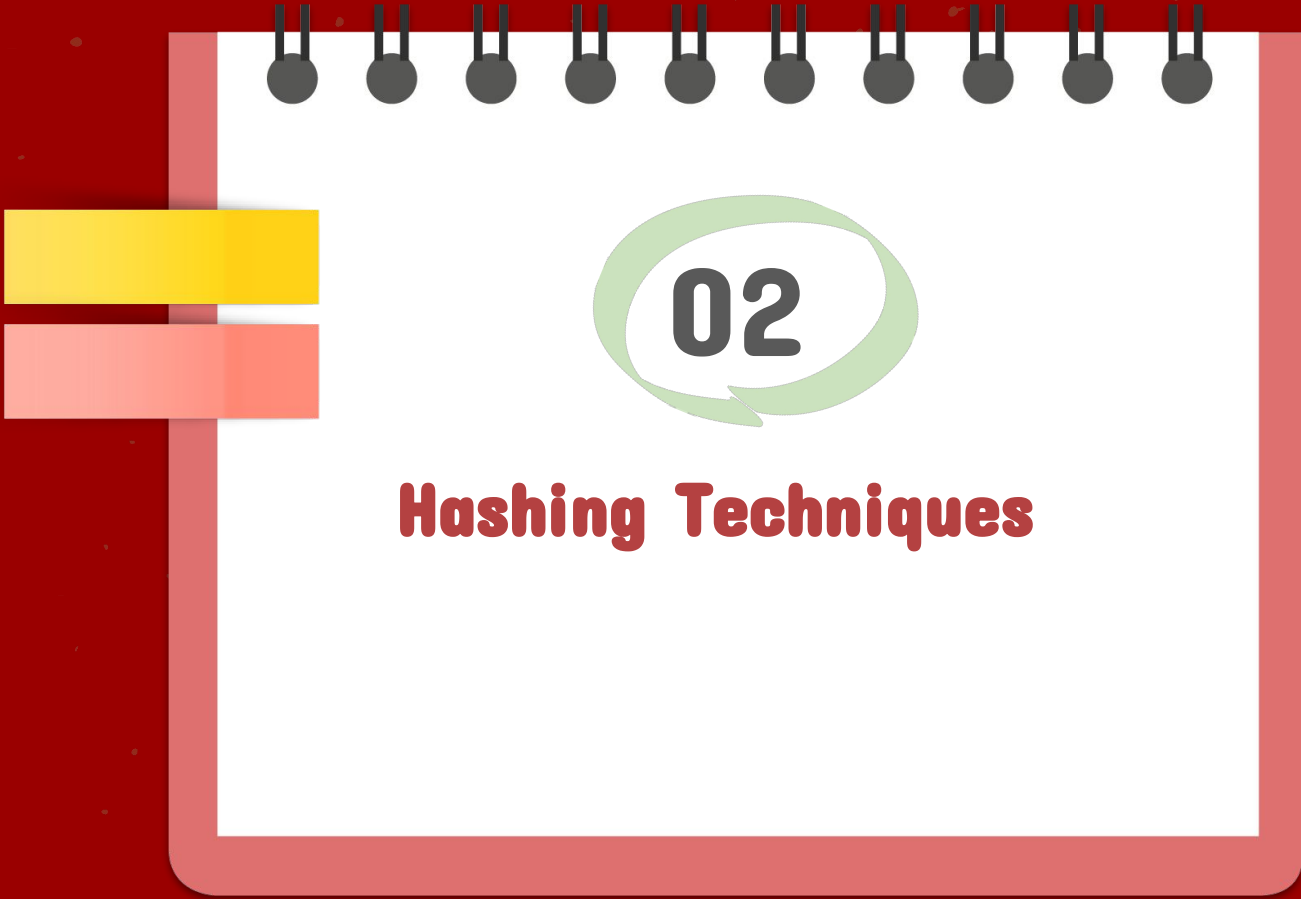
Collision can be reduced by allocating more file space than the minimum required to store the number of keys. However, this approach leads to wasted space.



Hashing

In hashing, there is a need to choose a good hash function and consequently, select a method to resolve, if not eliminate, collisions. A good hash function performs fast computation and produces less (or no) collisions.



A graphic of a spiral-bound notebook with a white page, a red cover, and a yellow and orange tab on the left. The page is titled '02 Hashing Techniques'.

02

Hashing Techniques

Simple Hash Techniques

Prime Number Division Method

This method is one of the most common randomizing scheme. If the key value is divided by a number n , the range of address generated will be of the form 0 to $n-1$. The formula is:

$$h(k) = k \bmod n$$

where k is the integer key value and n is a prime number

Rule of thumb:

Pick a prime number, close to a power of two, to be n



Simple Hash Techniques

Prime Number Division Method

If n is the total number of relative locations in the file, this method can be used to map the keys into n record locations.

n should be chosen to attempt to reduce number of collisions. If n is even, the result of the hash function has revealed an even value and if n is odd, the resulting value is also odd.



Simple Hash Techniques

Prime Number Division Method

Division with a prime number will not result to much collision, that is why it is the best choice for the divisor in this method.

We could choose a prime number that is close to the number of record positions in the file. However, this method may be used even if n is not a prime, but be prepared to handle more collisions.



Simple Hash Techniques

Prime Number Division Method

For example, let $n = 13$

Key Value k	Hash Value h(k)
125	8
845	0
444	2
256	9
345	7
745	4
902	5
569	10
254	7
382	5

Key Value k	Hash Value h(k)
234	0
431	2
947	11
981	6
792	12
459	4
725	10
652	2
421	5
458	3

In Java, to implement this hashing, it is as simple as:

```
int hash(int k, int n){  
    return (k % n);  
}
```



Simple Hash Techniques

Prime Number Division Method

Division with a prime number will not result to much collision, that is why it is the best choice for the divisor in this method. We could choose a prime number that is close to the number of record positions in the file. However, this method may be used even if n is not a prime, but be prepared to handle more collisions.



Simple Hash Techniques

In this technique, the key value is split into two or more parts and then added, ANDed, or XORed to get a hash address. If the resulting address has more digits than the highest address in the file, the excess high order digits are truncated.



Simple Hash Techniques

There are different ways of folding. A key value can be folded in half. This is ideal for relatively small key values since they would easily fit in the available addresses. If in any case, the key would be unevenly split, the left fold must be greater than the right fold.



Simple Hash Techniques

A key value can also be folded in thirds this is ideal for somewhat large key values. We can also fold alternate digits. The digits in the odd positions form one part, and the digits in the even positions form another.



Simple Hash Techniques

Folding in half and in thirds can be done in yet another two ways.

One is boundary folding where some parts of the folded keys are reversed (imitating the way we fold paper) and then summed.

Last is shift folding where no parts of the folded keys are reversed.



Simple Hash Techniques

The following are some examples of shift folding:

1. Even digits, folding in half
 $125758 \Rightarrow 125+758 \Rightarrow 883$
2. Folding in thirds
 $125758 \Rightarrow 12+57+58 \Rightarrow 127$
3. Odd digits , folding in half
 $7453212 \Rightarrow 7453+212 \Rightarrow 7665$
4. Different digits, folding in thirds
 $74532123 \Rightarrow 745+32+123 \Rightarrow 900$
5. Alternate digits
 $125758 \Rightarrow 155+278 \Rightarrow 433$



Simple Hash Techniques

The following are some examples of boundary folding:

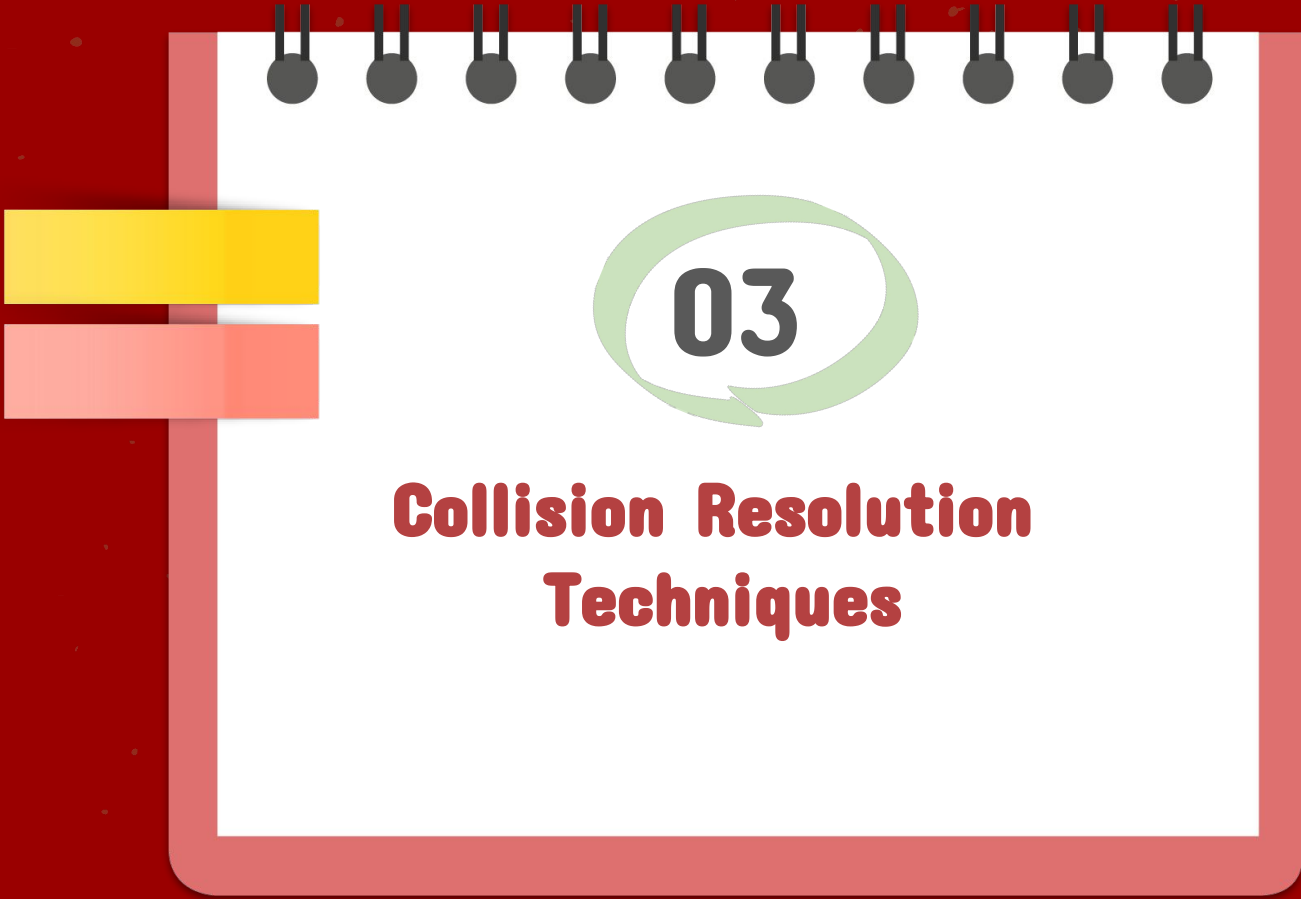
1. Even digits, folding in half
 $125758 \Rightarrow 125+857 \Rightarrow 982$
2. Folding in thirds
 $125758 \Rightarrow 21+57+85 \Rightarrow 163$
3. Odd digits , folding in half
 $7453212 \Rightarrow 7453+212 \Rightarrow 7665$
4. Different digits, folding in thirds
 $74532123 \Rightarrow 547+32+321 \Rightarrow 900$
5. Alternate digits
 $125758 \Rightarrow 155+872 \Rightarrow 1027$



Simple Hash Techniques

This method is useful for converting keys with large number of digits to a smaller number of digits so the address fits into a word memory. It is also easier to store since the keys do not require much space to be stored.





03

Collision Resolution Techniques

Collision Resolution Techniques

Choosing a good hashing algorithm based on how few collisions are likely to occur is the first step in avoiding collision. However, this will just minimize and will not eradicate the problem. To avoid collision, we could:

- *spread out records*, i.e., finding a hashing algorithm that distributes the records fairly randomly among the available addresses. However, it is hard to find a hashing algorithm that distributes the records evenly.
- *use extra memory*, if there are memory addresses to distribute the records into, it is easier to find a hashing algorithm than if we have almost equal number of addresses and records. One advantage is that the records are spread out evenly, eventually reducing collisions. However, this method wastes space.
- *use buckets*, i.e., put more than one record at a single address.

There are several collision resolution techniques and in this topic, we will cover chaining, use of buckets and open addressing.



Collision Resolution Techniques

Chaining

In chaining, m linked lists are maintained, one for each possible address in the hash table.



Collision Resolution Techniques

Chaining

Using chaining to resolve collision in storing the example in Prime Number Division Method of hashing:

Key Value k	Hash Value h(k)
125	8
845	0
444	2
256	9
345	7
745	4
902	5
569	10
254	7
382	5

Key Value k	Hash Value h(k)
234	0
431	2
947	11
981	6
792	12
459	4
725	10
652	2
421	5
458	3



Collision Resolution Techniques

Chaining

	KEY	LINK
0		Λ
1		Λ
2		Λ
3		Λ
4		Λ
5		Λ
6		Λ
7		Λ
8		Λ
9		Λ
10		Λ
11		Λ
12		Λ

Initial Table

	KEY	LINK
0	845	—
1		Λ
2	444	—
3	458	Λ
4	745	—
5	902	—
6	981	Λ
7	345	—
8	125	Λ
9	256	Λ
10	569	—
11	947	Λ
12	792	Λ

→ 234 Λ

→ 431 — → 652 Λ

→ 459 Λ

→ 382 — → 421 Λ

→ 254 Λ

→ 725 Λ

After Insertions

Keys **845** and **234** both hash to address 0 so they are linked together in the address. The case is the same for addresses 2, 4, 5, 7 and 10, while the rest of the addresses have no collision. The chaining method resolved the collision by providing additional link nodes to each of the values.



Collision Resolution Techniques

Use of Bucket

Like chaining, this method divides the hash table into m groups of records where each group contains exactly b records, having each address regarded as a bucket.

	KEY1	KEY2	KEY3
0	845	234	
1			
2	444	431	652
3	458		
4	745	459	
5	902	382	421
6	981		
7	345	254	
8	125		
9	256		
10	569	725	
11	947		
12	792		



Collision Resolution Techniques

Use of Bucket

Collision is redefined in this approach. It happens when a **bucket overflows – that is, when an insertion is attempted on a full bucket**. Hence, there is a significant reduction in the number of collisions. However, this method wastes some space and is not free from overflowing further, in which case an overflow policy must be invoked. In the above example, there are three slots in each address. Being static in size, problem arises when more than three values hash to a single address.

	KEY1	KEY2	KEY3
0	845	234	
1			
2	444	431	652
3	458		
4	745	459	
5	902	382	421
6	981		
7	345	254	
8	125		
9	256		
10	569	725	
11	947		
12	792		

Collision Resolution Techniques

Open Addressing (Probing)

In open addressing, when the address produced by a hash function $h(k)$ has no more space for insertion, an empty slot other than $h(k)$ in the hash table is located.

This process is called probing. In this empty slot, the new record that has collided with the old one, known as the colliding key, could be safely placed.

In this lesson, we will cover two techniques: linear probing and double hashing.



Collision Resolution Techniques

Open Addressing (Probing)

Linear Probing

Linear probing is one of the simplest techniques in handling collisions in which the file is scanned or probed sequentially as a circular file and the colliding key is stored in the nearest available space to the address.

This is used in systems where a "*first come, first served*" basis is followed.

An example is an airline reservation system where seats for chance passengers are given away when passengers for reserved seats don't arrive.



Collision Resolution Techniques

Open Addressing (Probing)

Linear Probing

Whenever a collision occurs at a certain address, the succeeding addresses are probed, that is, searched sequentially until an empty one is found. The key then uses this address. Array should be considered circular, so that when the last location is reached, the search proceeds to the first location of the array.



Collision Resolution Techniques

Open Addressing (Probing)

Linear Probing

The slots in a hash table may contain only one key or could also be a bucket. In this example, buckets of capacity 2 are used to store the following keys:

Key Value k	Hash Value h(k)
125	8
845	0
444	2
256	9
345	7
745	4
902	5
569	10
254	7
382	5

Key Value k	Hash Value h(k)
234	0
431	2
947	11
981	6
792	12
459	4
725	10
652	2
421	5
458	3



Collision Resolution Techniques

Open Addressing (Probing)

Linear Probing

	KEY1	KEY2
0	845	234
1		
2	444	431
3	652	458
4	745	459
5	902	382
6	981	421
7	345	254
8	125	
9	256	
10	569	725
11	947	
12	792	

Key Value k	Hash Value h(k)
125	8
845	0
444	2
256	9
345	7
745	4
902	5
569	10
254	7
382	5

Key Value k	Hash Value h(k)
234	0
431	2
947	11
981	6
792	12
459	4
725	10
652	2
421	5
458	3

This approach resolves the problem of overflowing in bucket addressing. Also, probing for available space makes the storage of overflowed keys near its home address in most cases. However, this method suffers from displacement problem where the keys that rightfully owns an address may be displaced by other keys that just probed to the said address.



Collision Resolution Techniques

Open Addressing (Probing)

Linear Probing

Example:

$$\text{hash}(k) = k \bmod 7$$

Here the table size $m = 7$

Note: 7 is a prime number.

0	
1	
2	
3	
4	
5	
6	

In **linear probing**, when there is a **collision**, we scan forwards for the the **next empty slot** (wrapping around when we reach the last slot).



Collision Resolution Techniques

Open Addressing (Probing)

Linear Probing

Example: Insert 18

$$\text{hash}(k) = k \bmod 7$$

$$\begin{aligned}\text{hash}(18) \\ &= 18 \bmod 7 \\ &= 4\end{aligned}$$

0	
1	
2	
3	
4	18
5	
6	



Collision Resolution Techniques

Open Addressing (Probing)

Linear Probing

Example: Insert 14

$$\text{hash}(k) = k \bmod 7$$

$$\begin{aligned}\text{hash}(14) \\ &= 14 \bmod 7 \\ &= 0\end{aligned}$$

0	14
1	
2	
3	
4	18
5	
6	



Collision Resolution Techniques

Open Addressing (Probing)

Linear Probing

Example: Insert 21

$$\text{hash}(k) = k \bmod 7$$

$$\begin{aligned}\text{hash}(21) \\ &= 21 \bmod 7 \\ &= 0\end{aligned}$$

0	14
1	21
2	
3	
4	18
5	
6	

Collision occurs!
Look for next empty slot.



Collision Resolution Techniques

Open Addressing (Probing)

Linear Probing

Example: Insert 1

$$\text{hash}(k) = k \bmod 7$$

$$\begin{aligned}\text{hash}(1) \\ &= 1 \bmod 7 \\ &= 1\end{aligned}$$

0	14
1	21
2	1
3	
4	18
5	
6	

Collides with 21
(hash value 0). Look
for **next empty slot**.



Collision Resolution Techniques

Open Addressing (Probing)

Linear Probing

Example: Insert 35

$$\text{hash}(k) = k \bmod 7$$

$$\begin{aligned}\text{hash}(35) \\ &= 35 \bmod 7 \\ &= 0\end{aligned}$$

0	14
1	21
2	1
3	35
4	18
5	
6	

Collision, need to check next 3 slots.



Collision Resolution Techniques

Open Addressing (Probing)

Linear Probing

Example: Find 35

$$\text{hash}(k) = k \bmod 7$$

$$\begin{aligned}\text{hash}(35) \\ &= 35 \bmod 7 \\ &= 0\end{aligned}$$

0	14
1	21
2	1
3	35
4	18
5	
6	

Found 35, after 4 probes.



Collision Resolution Techniques

Open Addressing (Probing)

Linear Probing

Example: Find 8

$$\text{hash}(k) = k \bmod 7$$

$$\begin{aligned}\text{hash}(8) &= 8 \bmod 7 \\ &= 1\end{aligned}$$

0	14
1	21
2	1
3	35
4	18
5	
6	

8 NOT found.
Need 5 probes!



Collision Resolution Techniques

Open Addressing (Probing)

Linear Probing

Example: Delete 21

$$\text{hash}(k) = k \bmod 7$$

$$\begin{aligned}\text{hash}(21) \\ &= 21 \bmod 7 \\ &= 0\end{aligned}$$

0	14
1	21
2	1
3	35
4	18
5	
6	



Collision Resolution Techniques

Open Addressing (Probing)

Linear Probing

Example: Delete 21

$$\text{hash}(k) = k \bmod 7$$

$$\begin{aligned}\text{hash}(21) \\ &= 21 \bmod 7 \\ &= 0\end{aligned}$$

0	14
1	21
2	1
3	35
4	18
5	
6	

Delete 21



Collision Resolution Techniques

Open Addressing (Probing)

Linear Probing

Example: Find 35

$$\text{hash}(k) = k \bmod 7$$

$$\begin{aligned}\text{hash}(35) \\ &= 35 \bmod 7 \\ &= 0\end{aligned}$$

0	14
1	
2	1
3	35
4	18
5	
6	

35 NOT found!
Incorrect!

We **cannot** simply
remove a value,
because it can
affect **find()**!



Collision Resolution Techniques

Open Addressing (Probing)

Linear Probing

How to Delete?

✓ Lazy Deletion

- Use three different states at each slot
 - Occupied
 - Deleted
 - Empty
- When a value is removed from linear probed hash table, we just mark the status of the slot as “deleted”, instead of emptying the slot
- Need to use a state array the same size as the hash table



Collision Resolution Techniques

Open Addressing (Probing)

Linear Probing

Example: Delete 21

$$\text{hash}(k) = k \bmod 7$$

$$\begin{aligned}\text{hash}(21) &= 21 \bmod 7 \\ &= 0\end{aligned}$$

0	14
1	21
2	1
3	35
4	18
5	
6	

Slot 1 is occupied
but now **marked**
as deleted.



Collision Resolution Techniques

Open Addressing (Probing)

Linear Probing

Example: Find 35

$$\text{hash}(k) = k \bmod 7$$

$$\begin{aligned}\text{hash}(35) &= 35 \bmod 7 \\ &= 0\end{aligned}$$

0	14
1	21
2	1
3	35
4	18
5	
6	

Found 35.
Now we can find 35.



Collision Resolution Techniques

Open Addressing (Probing)

Linear Probing

Example: Insert 15

$$\text{hash}(k) = k \bmod 7$$

$$\begin{aligned}\text{hash}(15) \\ &= 15 \bmod 7 \\ &= 1\end{aligned}$$

0	14
1	15
2	1
3	35
4	18
5	
6	

So, 15 is inserted into slot **1**, which was marked as deleted.

Note: We should insert a new value in **first** available slot so that the find operation for this value will be the fastest.



Collision Resolution Techniques

Open Addressing (Probing)

Double Hashing

Double hashing makes use of a second hash function, say $h_2(k)$, whenever there's a collision. The record is initially hashed to an address using the primary function. If the hashed address is not available, a second hash function is applied and added to the first hashed value, and the colliding key is hashed to the new address if there is available space. If there is none, the process is repeated.



Collision Resolution Techniques

Open Addressing (Probing)

Double Hashing

The following is the algorithm:

1. Use the primary hash function $h_1(k)$ to determine the position i at which to place the value.
2. If there is a collision, use the rehash function $r_h(i, k)$ successively until an empty slot is found:

$$r_h(i, k) = (i + h_2(k)) \bmod m$$

where m is the number of addresses



Collision Resolution Techniques

Open Addressing (Probing)

Double Hashing

Using the second hash function $h_2(k) = k \bmod 11$ in storing the following keys:

Key Value k	Hash Value $h(k)$
125	8
845	0
444	2
256	9
345	7
745	4
902	5
569	10
254	7
382	5

Key Value k	Hash Value $h(k)$
234	0
431	2
947	11
981	6
792	12
459	4
725	10
652	2
421	5
458	3



Collision Resolution Techniques

Open Addressing (Probing)

Double Hashing

For keys 125, 845, 444, 256, 345, 745, 902, 569, 254, 382, 234, 431, 947, 981, 792, 459 and 725, storage is pretty straightforward – no overflow happened.

	KEY1	KEY2
0	845	234
1		
2	444	431
3		
4	745	459
5	902	382
6	981	
7	345	254
8	125	
9	256	
10	569	725
11	947	
12	792	



Collision Resolution Techniques

Open Addressing (Probing)

Double Hashing

Inserting 652 into the hash table results to an overflow at address 2 so we rehash:

$$h_2(652) = 652 \bmod 11 = 3$$

$$r_h(2, 652) = (2 + 3) \bmod 13 = 5,$$

but address 5 is already full so rehash:

$$r_h(5, 652) = (5 + 3) \bmod 13 = 8, \text{ with space - so store here.}$$

Key Value k	Hash Value h(k)
125	8
845	0
444	2
256	9
345	7
745	4
902	5
569	10
254	7
382	5

Key Value k	Hash Value h(k)
234	0
431	2
947	11
981	6
792	12
459	4
725	10
652	2
421	5
458	3

	KEY1	KEY2
0	845	234
1		
2	444	431
3		
4	745	459
5	902	382
6	981	
7	345	254
8	125	652
9	256	
10	569	725
11	947	
12	792	

Collision Resolution Techniques

Open Addressing (Probing)

Double Hashing

Hashing 421 also results in a collision, so we rehash:

$$h_2(421) = 421 \bmod 11 = 3$$

$$r_h(5, 421) = (5 + 3) \bmod 13 = 8,$$

but address 8 is already full so rehash:

$$r_h(8, 421) = (8 + 3) \bmod 13 = 11, \text{ with space - so store here.}$$

Lastly, key 458 is stored at address 3.

Key Value k	Hash Value h(k)
125	8
845	0
444	2
256	9
345	7
745	4
902	5
569	10
254	7
382	5

Key Value k	Hash Value h(k)
234	0
431	2
947	11
981	6
792	12
459	4
725	10
652	2
421	5
458	3

	KEY1	KEY2
0	845	234
1		
2	444	431
3	458	
4	745	459
5	902	382
6	981	
7	345	254
8	125	652
9	256	
10	569	725
11	947	421
12	792	